Universidad de Carabobo

Simulador de Caché

Correspondencia Directa y Asociativa por Vías

Análisis de Rendimiento y Comparativa de Configuraciones

Datos Académicos	
Asignatura	Arquitectura del Computador

Equipo de Trabajo		
Nombre	**Cédula**	**Email**
Alessandro Ocanto	31467550	alessandro4876926@gmail.com
Abraham Quintero	30958868	abraham1gamer@gmail.com

4 de abril de 2026

4 de abril de 2026

Índice

1. Introducción **3**
1.1. Motivación . . . 3
1.2. Objetivos del Proyecto . . . 3

2. Fundamentos Teóricos **3**
2.1. Principio de Localidad . . . 3
2.2. Jerarquía de Memoria . . . 4
2.3. Organización de la Caché . . . 4
2.3.1. Caché de Correspondencia Directa . . . 4
2.3.2. Caché Asociativa por Conjuntos . . . 4
2.4. Estructura de la Dirección . . . 5
2.5. Modelo de las Tres C (3Cs) . . . 5
2.6. Políticas de Reemplazo . . . 5
2.7. Políticas de Escritura . . . 6
2.8. Técnica de Prefetching . . . 6
2.9. Métrica de Rendimiento: AMAT . . . 6

3. Implementación del Simulador **6**
3.1. Arquitectura del Software . . . 6
3.2. Estructura de la Línea de Caché . . . 7
3.3. Cálculo de Mapeo . . . 7
3.4. Política de Reemplazo LRU . . . 7
3.5. Mecanismo de Prefetching . . . 8
3.6. ¿Por qué usar direcciones hexadecimales? . . . 8

4. Configuración Experimental **9**
4.1. Parámetros de Simulación . . . 9
4.2. Patrones de Prueba . . . 9
4.2.1. Patrón 1: Localidad Espacial . . . 9
4.2.2. Patrón 2: Conflicto . . . 9
4.2.3. Patrón 3: Reuso de Datos . . . 9

5. Resultados Experimentales **9**
5.1. Patrón 1: Localidad Espacial . . . 9
5.2. Patrón 2: Conflicto . . . 10
5.3. Patrón 3: Reuso de Datos . . . 10
5.4. Resumen Comparativo . . . 11

6. Análisis y Discusión **11**
6.1. Impacto de la Asociatividad 11
6.2. Efecto del Prefetching 12
6.3. Clasificación de Fallos (3 Cs) 12
6.4. Localidad Espacial y Tamaño de Bloque 12
6.5. Interpretación de la Tabla de Seguimiento 12

7. Conclusiones **13**
7.1. Trabajo Futuro 13

A. Anexo: Código Completo del Simulador **13**

1. Introducción

1.1. Motivación

La memoria caché es uno de los componentes más críticos en la arquitectura de computadores modernos. Como señalan Patterson y Hennessy en *Estructura y Diseño de Computadores* [1], el principio de localidad —tanto temporal como espacial— es fundamental para el funcionamiento eficiente de las jerarquías de memoria. Sin una caché adecuadamente diseñada, los procesadores modernos, que pueden ejecutar miles de millones de instrucciones por segundo, pasarían la mayor parte de su tiempo esperando datos de la memoria principal.

La brecha entre la velocidad del procesador y la memoria principal ha crecido dramáticamente en las últimas décadas. Mientras que los procesadores han experimentado mejoras exponenciales en frecuencia de reloj, la latencia de la DRAM ha mejorado a un ritmo mucho más lento. Esta disparidad ha hecho que las memorias caché sean cada vez más importantes para el rendimiento general del sistema.

Cuadro 1: Brecha procesador-memoria a lo largo del tiempo

Año	**Procesador**	**Frecuencia**	**Latencia DRAM**
1985	Intel 80386	16 MHz	150 ns
1995	Intel Pentium Pro	200 MHz	60 ns
2005	Intel Pentium 4	3.6 GHz	50 ns
2015	Intel Core i7	4.0 GHz	40 ns
2024	AMD Ryzen 9	5.7 GHz	35 ns

1.2. Objetivos del Proyecto

1. Implementar un simulador de caché configurable para diferentes niveles de asociatividad
2. Analizar el impacto del mapeo directo frente al asociativo por vías
3. Evaluar el efecto del prefetching en patrones de acceso secuencial
4. Comparar cuantitativamente las tasas de aciertos y fallos entre configuraciones
5. Demostrar los principios de localidad espacial y temporal mediante experimentos controlados

2. Fundamentos Teóricos

2.1. Principio de Localidad

El principio de localidad es la base del funcionamiento de las memorias caché. Patterson y Hennessy [1] establecen que:

> *"Los programas acceden a una parte relativamente pequeña del espacio de direcciones en un determinado instante."*

Existen dos tipos de localidad:

Cuadro 2: Tipos de localidad

Tipo	**Definición**	**Ejemplo**
Localidad Temporal	Si se accede a una dirección, es probable que se acceda de nuevo pronto	Lazos, variables contadoras, acumuladores
Localidad Espacial	Si se accede a una dirección, es probable que se accedan a direcciones cercanas	Acceso secuencial a arrays, ejecución lineal de código

2.2. Jerarquía de Memoria

La jerarquía de memoria aprovecha el principio de localidad para ofrecer la ilusión de una memoria grande y rápida. A continuación se muestra la organización típica:

```
Nivel 1: CPU (registros)          - Tamaño: ~1 KB      - Tiempo: ~0.3 ns
Nivel 2: Cache L1                 - Tamaño: 32-64 KB   - Tiempo: ~1 ns
Nivel 3: Cache L2                 - Tamaño: 256 KB-8MB - Tiempo: ~10 ns
Nivel 4: Memoria DRAM             - Tamaño: 4-64 GB    - Tiempo: ~100 ns
Nivel 5: Disco                    - Tamaño: 1-20 TB    - Tiempo: ~10 ms
```

figureJerarquía de memoria típica (tiempos aproximados en 2024)

2.3. Organización de la Caché

2.3.1. Caché de Correspondencia Directa

En una caché de correspondencia directa, cada bloque de memoria principal tiene **una única ubicación posible** en la caché. La posición viene determinada por:

$$\boxed{\text{Índice} = (\text{Dirección_bloque}) \mod (\text{Número_de_bloques_en_cache})} \tag{1}$$

- **Ventaja:** Implementación simple y rápida, menor consumo energético
- **Desventaja:** Susceptible a conflictos cuando múltiples direcciones acceden al mismo índice

2.3.2. Caché Asociativa por Conjuntos

En una caché asociativa por conjuntos de N vías, cada bloque de memoria puede ubicarse en **cualquiera de las N posiciones** de un conjunto determinado por:

$$\boxed{\text{Conjunto} = (\text{Dirección_bloque}) \mod (\text{Número_de_conjuntos})} \tag{2}$$

- **Ventaja:** Reduce los fallos de conflicto, mejor tasa de aciertos
- **Desventaja:** Mayor complejidad hardware, mayor tiempo de acceso, mayor consumo energético

2.4. Estructura de la Dirección

La dirección de memoria se descompone en tres campos:

```
+-------------------+-------------------+-------------------+
|       Tag         |      Índice       |   Desplazamiento  |
|    (bits altos)   |   (bits medios)   |    (bits bajos)   |
+-------------------+-------------------+-------------------+
```

figureDescomposición de la dirección de memoria

- **Tag (Etiqueta):** Identifica qué bloque de memoria está almacenado
- **Índice:** Selecciona el conjunto/camino a examinar
- **Desplazamiento:** Selecciona la palabra dentro del bloque

2.5. Modelo de las Tres C (3Cs)

Patterson y Hennessy [1] clasifican los fallos de caché en tres categorías fundamentales:

Cuadro 3: Modelo de las 3 Cs

Tipo	**Causa**	**Solución**
Compulsorios (cold)	Primer acceso al bloque	Aumentar tamaño de bloque
Capacidad	Caché insuficiente para el conjunto de trabajo	Aumentar tamaño de caché
Conflicto	Colisiones en el mapeo	Aumentar asociatividad

2.6. Políticas de Reemplazo

Cuadro 4: Comparación de políticas de reemplazo

Política	**Descripción**	**Ventajas/Desventajas**
LRU (Least Recently Used)	Reemplaza el bloque no usado por más tiempo	Buena correlación con localidad temporal, costosa de implementar
FIFO (First In First Out)	Reemplaza el bloque más antiguo	Simple, pero no considera frecuencia de uso
Aleatorio	Selecciona un bloque aleatorio	Muy simple, evita comportamientos patológicos

2.7. Políticas de Escritura

Cuadro 5: Políticas de escritura en caché

Política	**Descripción**	**Características**
Write-Through	Escribe simultáneamente en caché y memoria	Consistencia garantizada, mayor tráfico de bus
Write-Back	Escribe solo en caché, marca bloque como sucio	Menor tráfico, necesita bit de consistencia
Write-Allocate	En fallo de escritura, carga el bloque en caché	Aprovecha futuros accesos
No-Write-Allocate	En fallo de escritura, escribe directamente en memoria	Evita cargas innecesarias

2.8. Técnica de Prefetching

El prefetching consiste en anticipar la carga de datos a la caché antes de que sean solicitados explícitamente. Esta técnica explota la localidad espacial para reducir la penalización por fallos.

$$\boxed{\text{Dirección_prefetch} = \text{Dirección_actual} + \text{Tamaño_bloque}} \tag{3}$$

2.9. Métrica de Rendimiento: AMAT

El Tiempo Medio de Acceso a Memoria (Average Memory Access Time, AMAT) es la métrica fundamental para evaluar el rendimiento de una jerarquía de caché:

$$\boxed{\text{AMAT} = \text{Tiempo_acierto} + \text{Tasa_fallos} \times \text{Penalización_fallo}} \tag{4}$$

3. Implementación del Simulador

3.1. Arquitectura del Software

El simulador se ha implementado en C++ utilizando la STL (Standard Template Library). La estructura principal es la clase CacheSimulator:

```
class CacheSimulator {
private:
    unsigned int cacheSize;      // Tama o total de la cach  (bytes)
    unsigned int blockSize;      // Tama o de bloque (bytes)
    unsigned int ways;           // Grado de asociatividad
    unsigned int numSets;        // N mero de conjuntos
    vector<list<CacheLine>> cache; // Estructura de la cach
    long long hits, misses;      // Contadores de estad sticas

public:
    CacheSimulator(unsigned int cSize, unsigned int bSize, unsigned int
   w);
    bool access(unsigned int address, bool enablePrefetch);
    void prefetch(unsigned int address);
    void printFinalStats(string mode);
};
```

Listing 1: Clase principal del simulador

3.2. Estructura de la Línea de Caché

```
struct CacheLine {
    bool valid;           // Bit de validez (indica si la l nea
   contiene datos)
    unsigned int tag;     // Etiqueta (identificador del bloque)
    CacheLine(bool v, unsigned int t) : valid(v), tag(t) {}
};
```

Listing 2: Estructura CacheLine

3.3. Cálculo de Mapeo

El cálculo de la posición en caché sigue la fórmula estándar presentada en la bibliografía:

```
unsigned int blockAddr = address / blockSize;      // Direcci n de
   bloque
unsigned int setIndex = blockAddr % numSets;       //  ndice  del
   conjunto
unsigned int tag = blockAddr / numSets;            // Etiqueta
```

Listing 3: Cálculo de índice y tag

3.4. Política de Reemplazo LRU

La política LRU (Least Recently Used) se implementa manteniendo los bloques más recientemente usados al frente de la lista:

```
// Buscar el bloque en el conjunto
for (auto it = currentSet.begin(); it != currentSet.end(); ++it) {
    if (it->valid && it->tag == tag) {
        hits++;
        found = true;
        // LRU: mover al frente (MRU)
        CacheLine foundLine = *it;
        currentSet.erase(it);
        currentSet.push_front(foundLine);
        break;
    }
}

if (!found) {
    misses++;
    if (currentSet.size() >= (size_t)ways) currentSet.pop_back();
    currentSet.push_front(CacheLine(true, tag));
}
```

Listing 4: Implementación de LRU

3.5. Mecanismo de Prefetching

```
void prefetch(unsigned int address) {
    unsigned int nextAddr = address + blockSize;
    unsigned int blockAddr = nextAddr / blockSize;
    unsigned int setIndex = blockAddr % numSets;
    unsigned int tag = blockAddr / numSets;

    list<CacheLine>& currentSet = cache[setIndex];

    // Verificar si ya est  en cach
    for (auto it = currentSet.begin(); it != currentSet.end(); ++it) {
        if (it->valid && it->tag == tag) return;
    }

    // Si no est , cargarlo (con reemplazo LRU si es necesario)
    if (currentSet.size() >= (size_t)ways) currentSet.pop_back();
    currentSet.push_front(CacheLine(true, tag));
}
```

Listing 5: Implementación del prefetching

3.6. ¿Por qué usar direcciones hexadecimales?

Las direcciones se representan en hexadecimal (0x100, 0x104, etc.) por varias razones fundamentales:

1. **Compactación:** Un dígito hexadecimal representa 4 bits, lo que hace que las direcciones sean mucho más legibles que su equivalente binario:
   - Binario: 0001 0000 0000 (12 caracteres)
   - Hexadecimal: 0x100 (5 caracteres)
2. **Alineación natural:** Los tamaños de bloque suelen ser potencias de 2 (16 bytes = 2^4), lo que hace que las direcciones hexadecimales muestren claramente los límites de bloque:
   - 0x100 a 0x10F $\rightarrow$ mismo bloque
   - 0x110 $\rightarrow$ bloque siguiente
3. **Conversión sencilla:** La correspondencia entre hexadecimal y binario es directa (4 bits $\rightarrow$ 1 hex), facilitando la extracción manual de campos (tag, índice, desplazamiento)
4. **Convención estándar:** Los lenguajes de programación (C, C++, Java, Python) y las herramientas de depuración utilizan hexadecimal para direcciones de memoria

4. Configuración Experimental

4.1. Parámetros de Simulación

Cuadro 6: Configuración del hardware simulado

Parámetro	Valor
Tamaño de caché	1024 bytes (1 KB)
Tamaño de bloque	16 bytes
Número de conjuntos (directa)	64
Número de conjuntos (2 vías)	32
Número de conjuntos (4 vías)	16

4.2. Patrones de Prueba

Se han diseñado tres patrones de acceso para evaluar diferentes aspectos del comportamiento de la caché:

4.2.1. Patrón 1: Localidad Espacial

```
Direcciones: 0x000, 0x004, 0x008, 0x00C, 0x010, 0x014, ..., 0x02C
```

Este patrón evalúa la capacidad de la caché para aprovechar la localidad espacial y la efectividad del prefetching en accesos secuenciales.

4.2.2. Patrón 2: Conflicto

```
Direcciones: 0x000, 0x400, 0x800, 0x000, 0x400, 0x800, 0x000, 0x400, 0x800
```

Este patrón fue diseñado específicamente para crear conflictos de mapeo y demostrar las ventajas de la asociatividad.

4.2.3. Patrón 3: Reuso de Datos

```
Direcciones: 0x100, 0x104, 0x108, 0x500, 0x504,
             0x100, 0x104, 0x108, 0x500, 0x504
```

Este patrón evalúa la localidad temporal y la capacidad de la caché para retener datos entre accesos.

5. Resultados Experimentales

5.1. Patrón 1: Localidad Espacial

Cuadro 7: Resultados - Patrón de localidad espacial

Configuración	Hits	Misses	Hit Rate
Mapeo Directo	9	3	75.00 %
Asociativo 2 vías	9	3	75.00 %
Asociativo 4 vías	9	3	75.00 %
Asociativo 4 vías + Prefetch	11	1	91.67 %

Análisis: Con bloque de 16 bytes, las direcciones 0x000, 0x004, 0x008, 0x00C pertenecen al mismo bloque. El primer acceso (0x000) genera un fallo obligatorio, pero los tres siguientes son aciertos. Este comportamiento demuestra el **principio de localidad espacial**.

```
Tiempo ->
Bloque 0x000-0x00F: [MISS] [HIT] [HIT] [HIT]
Bloque 0x010-0x01F:                 [PF]  [HIT] [HIT] [HIT]
                                          |
                                          +-- Prefetch activado
```

figureEfecto del prefetching en accesos secuenciales

5.2. Patrón 2: Conflicto

Cuadro 8: Resultados - Patrón de conflicto

Configuración	**Hits**	**Misses**	**Hit Rate**
Mapeo Directo	0	9	0.00 %
Asociativo 2 vías	0	9	0.00 %
Asociativo 4 vías	6	3	66.67 %
Asociativo 4 vías + Prefetch	6	3	66.67 %

Análisis del conflicto:

$$\begin{aligned}
\text{Tamaño bloque} &= 16 \text{ bytes} \\
\text{Direcciones de bloque} &= 0x00, 0x40, 0x80 \\
\text{numSets (directa)} &= 1024/16/1 = 64 \\
\text{Índice} &= \text{bloque} \mod 64 = 0 \text{ para todas} \\
\text{numSets (4 vías)} &= 1024/16/4 = 16 \\
\text{Conjunto} &= \text{bloque} \mod 16 = 0 \text{ para todas}
\end{aligned} \tag{5}$$

En mapeo directo, las tres direcciones compiten por el mismo índice, expulsándose mutuamente (0 % aciertos). En la caché de 4 vías, las tres pueden coexistir, logrando aciertos en las iteraciones 4-9.

5.3. Patrón 3: Reuso de Datos

Cuadro 9: Resultados - Patrón de reuso

Configuración	**Hits**	**Misses**	**Hit Rate**
Mapeo Directo	6	4	60.00 %
Asociativo 2 vías	8	2	80.00 %
Asociativo 4 vías	8	2	80.00 %
Asociativo 4 vías + Prefetch	8	2	80.00 %

Análisis: Los primeros 5 accesos son fallos obligatorios (compulsorios). Los últimos 5 deberían ser aciertos si los datos persisten en caché. La caché de mapeo directo falla en 0x100 y 0x500 en la segunda iteración debido a conflictos, mientras que las cachés asociativas retienen ambos bloques.

5.4. Resumen Comparativo

Cuadro 10: Resumen de tasas de aciertos por configuración

Configuración	Espacial	Conflicto	Reuso
Mapeo Directo	75.00 %	0.00 %	60.00 %
Asociativo 2 vías	75.00 %	0.00 %	80.00 %
Asociativo 4 vías	75.00 %	66.67 %	80.00 %
4 vías + Prefetch	91.67 %	66.67 %	80.00 %

```
Hit Rate (%)
   100 +
    90 +                          ****
    80 +              ****  ****  ****
    70 +        ****  ****  ****  ****
    60 +  ****  ****  ****  ****  ****
    50 +  ****  ****  ****  ****  ****
    40 +  ****  ****  ****  ****  ****
    30 +  ****  ****  ****  ****  ****
    20 +  ****  ****  ****  ****  ****
    10 +  ****  ****  ****  ****  ****
     0 +----+-----+-----+----------+
         Dir.  2vias 4vias 4vias+PF

         ## Espacial  ## Conflicto  ## Reuso
```

figureComparativa de tasas de aciertos

6. Análisis y Discusión

6.1. Impacto de la Asociatividad

Los resultados demuestran que aumentar la asociatividad mejora significativamente la tasa de aciertos en patrones de conflicto, pero tiene un impacto limitado en patrones con buena localidad espacial. La caché de 4 vías logra un 66.67 % de aciertos en el patrón de conflicto, frente al 0 % de la caché directa.

$$\text{Mejora} = \frac{66{,}67\,\% - 0\,\%}{0\,\%} \times 100\,\% = \infty \text{ (en este caso)} \tag{6}$$

Sin embargo, este aumento en rendimiento tiene un coste en términos de hardware y tiempo de acceso:

- **Caché directa:** 1 comparador, acceso más rápido
- **Caché 4 vías:** 4 comparadores, multiplexor 4:1, mayor consumo

6.2. Efecto del Prefetching

El prefetching es altamente efectivo en patrones secuenciales (91.67 % vs 75 %), representando una mejora del 22.2 % en la tasa de aciertos. Sin embargo, no aporta beneficio en patrones de acceso aleatorio o de conflicto, ya que el prefetching asume localidad espacial (el bloque siguiente será solicitado).

$$\text{Mejora con prefetch} = \frac{91{,}67 - 75}{75} \times 100\,\% = 22{,}2\,\% \tag{7}$$

6.3. Clasificación de Fallos (3 Cs)

Aplicando el modelo de las 3 Cs a los resultados obtenidos:

Cuadro 11: Clasificación de fallos por patrón

Patrón	**Compulsorios**	**Capacidad**	**Conflicto**
Localidad Espacial	Altos	Bajos	Muy bajos
Conflicto	Bajos	Bajos	Muy altos (en directa)
Reuso	Medios	Medios	Medios

6.4. Localidad Espacial y Tamaño de Bloque

El uso de bloques de 16 bytes permite explotar la localidad espacial: un solo fallo carga 4 palabras (16 bytes), beneficiando los accesos a direcciones contiguas. Como señalan Patterson y Hennessy [1]:

> *"Las caches aprovechan la localidad espacial transmitiendo hacia niveles superiores de la jerarquía bloques de varias palabras que se almacenan en posiciones contiguas de memoria."*

6.5. Interpretación de la Tabla de Seguimiento

A continuación se muestra la salida del simulador para el patrón de localidad espacial:

```
TABLA DE SEGUIMIENTO PASO A PASO
Direccion   Directo     Asoc-2      Asoc-4      Asoc-4+PF
------------------------------------------------------------
0x0         MISS        MISS        MISS        MISS
0x4         HIT         HIT         HIT         HIT
0x8         HIT         HIT         HIT         HIT
0xc         HIT         HIT         HIT         HIT
0x10        MISS        MISS        MISS        HIT
0x14        HIT         HIT         HIT         HIT
...
```

Se observa que:

1. El primer acceso a cada bloque siempre es MISS (fallo compulsorio)
2. Los accesos siguientes dentro del mismo bloque son HIT (localidad espacial)
3. El prefetching convierte el primer acceso al bloque 0x010 en HIT

7. Conclusiones

El simulador implementado demuestra exitosamente los conceptos fundamentales de las memorias caché:

1. **El mapeo directo** es simple pero vulnerable a conflictos
2. **La asociatividad** reduce los fallos de conflicto a costa de mayor complejidad
3. **El prefetching** mejora significativamente el rendimiento en patrones secuenciales
4. **La localidad espacial** es fundamental para la efectividad de la caché

Las tasas de aciertos obtenidas (0 % a 91.67 %) muestran la importancia de seleccionar la configuración adecuada según el patrón de acceso de la aplicación.

7.1. Trabajo Futuro

- Implementar cachés multinivel (L1/L2/L3)
- Agregar política de reemplazo FIFO y aleatoria
- Modelar latencias de acceso para calcular AMAT
- Implementar prefetching adaptativo (stride prediction)
- Interfaz gráfica para visualización del estado de la caché
- Simular diferentes tamaños de bloque y caché

Referencias

[1] Patterson, D. A., & Hennessy, J. L. (2011). *Estructura y Diseño de Computadores: La Interfaz Hardware/Software* (4ª ed.). Reverté.

[2] Hennessy, J. L., & Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann.

[3] Smith, A. J. (1982). "Cache Memories". *ACM Computing Surveys*, 14(3), 473-530.

[4] Amdahl, G. M. (1967). "Validity of the single processor approach to achieving large scale computing capabilities". *AFIPS Conference Proceedings*, 30, 483-485.

A. Anexo: Código Completo del Simulador

```
#include <iostream>
#include <vector>
#include <list>
#include <iomanip>
#include <fstream>
#include <string>

```

```
using namespace std;

/**
 * Estructura que representa una l nea de cach
 */
struct CacheLine {
    bool valid;           // Bit de validez
    unsigned int tag;     // Etiqueta
    CacheLine(bool v, unsigned int t) : valid(v), tag(t) {}
};

/**
 * Simulador de cach  configurable
 */
class CacheSimulator {
private:
    unsigned int cacheSize, blockSize, ways, numSets;
    vector<list<CacheLine>> cache;
    long long hits, misses;

public:
    CacheSimulator(unsigned int cSize, unsigned int bSize, unsigned int w)
        : cacheSize(cSize), blockSize(bSize), ways(w), hits(0), misses(0) {
        numSets = cacheSize / (blockSize * ways);
        cache.resize(numSets);
    }

    void prefetch(unsigned int address) {
        unsigned int nextAddr = address + blockSize;
        unsigned int blockAddr = nextAddr / blockSize;
        unsigned int setIndex = blockAddr % numSets;
        unsigned int tag = blockAddr / numSets;

        list<CacheLine>& currentSet = cache[setIndex];

        for (auto it = currentSet.begin(); it != currentSet.end(); ++it) {
            if (it->valid && it->tag == tag) return;
        }

        if (currentSet.size() >= (size_t)ways) currentSet.pop_back();
        currentSet.push_front(CacheLine(true, tag));
    }

    bool access(unsigned int address, bool enablePrefetch) {
        unsigned int blockAddr = address / blockSize;
        unsigned int setIndex = blockAddr % numSets;
        unsigned int tag = blockAddr / numSets;

        list<CacheLine>& currentSet = cache[setIndex];
        bool found = false;

        for (auto it = currentSet.begin(); it != currentSet.end(); ++it) {
            if (it->valid && it->tag == tag) {
                hits++;
```

```
                found = true;
                CacheLine foundLine = *it;
                currentSet.erase(it);
                currentSet.push_front(foundLine);
                break;
            }
        }

        if (!found) {
            misses++;
            if (currentSet.size() >= (size_t)ways) currentSet.pop_back();
            currentSet.push_front(CacheLine(true, tag));
        }

        if (enablePrefetch) prefetch(address);
        return found;
    }

    void printFinalStats(string mode) {
        double total = hits + misses;
        double hitRate = (total > 0) ? (hits / total) * 100 : 0;

        cout << left << setw(28) << mode << " | "
             << "Hits: " << setw(4) << hits << " | "
             << "Misses: " << setw(4) << misses << " | "
             << "Hit Rate: " << fixed << setprecision(2) << hitRate << "\%" << endl;
    }
};

int main() {
    unsigned int cSize = 1024;
    unsigned int bSize = 16;

    ifstream archivo("direcciones.txt");
    if (!archivo.is_open()) {
        cerr << "Error: No se pudo abrir direcciones.txt" << endl;
        return 1;
    }

    vector<unsigned int> trace;
    unsigned int addr;
    while (archivo >> hex >> addr) trace.push_back(addr);
    archivo.close();

    CacheSimulator dm(cSize, bSize, 1);
    CacheSimulator sa2(cSize, bSize, 2);
    CacheSimulator sa4(cSize, bSize, 4);
    CacheSimulator sa4pf(cSize, bSize, 4);

    cout << "\nTABLA DE SEGUIMIENTO PASO A PASO" << endl;
    cout << left << setw(12) << "Direccion"
         << setw(12) << "Directo"
         << setw(12) << "Asoc-2"
         << setw(12) << "Asoc-4"
         << setw(12) << "Asoc-4+PF" << endl;
    cout << string(60, '-') << endl;
```

```

    for (size_t i = 0; i < trace.size(); ++i) {
        bool rDM = dm.access(trace[i], false);
        bool rSA2 = sa2.access(trace[i], false);
        bool rSA4 = sa4.access(trace[i], false);
        bool rSA4PF = sa4pf.access(trace[i], true);

        cout << "0x" << hex << left << setw(10) << trace[i]
             << setw(12) << (rDM ? "HIT" : "MISS")
             << setw(12) << (rSA2 ? "HIT" : "MISS")
             << setw(12) << (rSA4 ? "HIT" : "MISS")
             << setw(12) << (rSA4PF ? "HIT" : "MISS") << dec << endl;
    }

    cout << "\n" << string(75, '=') << endl;
    cout << "                    ANALISIS COMPARATIVO FINAL" << endl;
    cout << string(75, '=') << endl;
    dm.printFinalStats("Mapeo Directo");
    sa2.printFinalStats("Asociativo 2 Vias");
    sa4.printFinalStats("Asociativo 4 Vias");
    sa4pf.printFinalStats("Asociativo 4 Vias + Prefetch");
    cout << string(75, '=') << endl;

    return 0;
}
```

Listing 6: Código completo del simulador de caché